

API TESTING MASTERCLASS

Newman & CI/CD Automation Guide

Run your Postman collections from the command line and in CI.

What Newman Is

Newman is Postman's command-line collection runner. It executes the exact same collections, environments and tests you build in the Postman app, but headlessly — which means your API checks can run on a build server, on a schedule, or as a pre-deployment gate rather than only when someone clicks Send. If you can run a collection in Postman, you can run it in CI with Newman.

Installing Newman

Newman is distributed as an npm package and needs Node.js (LTS) installed first. Install it globally so it is on your PATH: `npm i -g newman`

- Check the install worked: `newman --version`
- For the rich HTML report, also install the reporter: `npm i -g newman-reporter-htmlextra`
- On CI it is often cleaner to install locally (`npm i -D newman`) and call it via `npx newman`, so the version is pinned in `package.json`.

Running a Collection with an Environment

Export your collection and environment from Postman (as v2.1 JSON), then run: `newman run collection.json -e env.json` — the `-e` flag supplies the environment file so variables like `{{baseUrl}}` and `{{token}}` resolve exactly as they do in the app.

- Keep secrets out of the committed environment file — inject tokens at run time (see the CI section).
- You can point at a shared collection by URL instead of a file if you use Postman's API.
- Use `--folder "Smoke"` to run only a named folder for a fast subset.

Data-Driven Runs

To run the same requests against many rows of input, supply a data file with `-d`: `newman run collection.json -e env.json -d data.csv` — Newman executes every request once per row, substituting `{{columnName}}` variables from the file. Both CSV and JSON data files are supported.

- The CSV header row names the variables you reference in the request as `{{header}}`.
- Add an `iterationData-aware` assertion so each row asserts against its own expected value.
- Great for boundary and negative data sets without duplicating requests.

Reporters

Reporters control how results are output. Select one or more with `--reporters` and configure them with reporter-specific flags.

Reporter	Flag	Best for
cli	<code>--reporters cli</code>	Human-readable summary in the terminal / build log (default)
json	<code>--reporters json --reporter-json-export out.json</code>	Machine-readable results for dashboards or further processing
junit	<code>--reporters junit --reporter-junit-export results.xml</code>	CI test-result panels that expect JUnit XML
htmlextra	<code>--reporters htmlextra --reporter-htmlextra-export report.html</code>	A rich, shareable HTML report for stakeholders

You can combine them, e.g. `--reporters cli,htmlextra,junit` — the CLI keeps the build log readable while the HTML and JUnit files become artifacts.

Exit Codes (and Why CI Cares)

Newman returns exit code 0 when every request and assertion passes, and a non-zero code when anything fails. CI systems read that exit code to decide whether a step passed — so a failing API test automatically fails the pipeline and blocks a bad build, without anyone reading the log.

- Do not swallow the exit code (e.g. avoid trailing `"|| true"`) or failures will pass silently.
- Use `--bail` to stop on the first failure when you want fast feedback.
- Treat a non-zero Newman exit as a genuine gate, not a warning to ignore.

BEST PRACTICE

Let the exit code do the gating. A green pipeline should mean every API assertion passed; a red one should mean a real problem. Never mask Newman's exit code just to keep the build green.

Running in GitHub Actions

A GitHub Actions workflow is a YAML file in `.github/workflows/`. In plain terms, the job runs these steps in order:

1. Checkout — use `actions/checkout` to pull your repository (including the exported collection and environment files) onto the runner.
2. Set up Node — use `actions/setup-node` with an LTS version so `npm` and `npx` are available.
3. Install Newman — run `npm i -g newman newman-reporter-htmlextra` (or `npm ci` if it is a dev dependency).
4. Run the collection — run `newman run collection.json -e env.json --reporters cli,htmlextra,junit --reporter-htmlextra-export report.html --reporter-junit-export results.xml`, injecting secrets via `--env-var "token=${{ secrets.API_TOKEN }}"`.
5. Upload the report — use `actions/upload-artifact` to attach `report.html` and `results.xml` so anyone can download and view them from the run.

Because the Run step fails the job on a non-zero exit code, the pipeline blocks automatically when an assertion fails — the report artifact is still uploaded so you can see exactly what broke.

Scheduling & Monitoring

- Run smoke collections on a schedule (a cron trigger in your CI) to catch environment drift and third-party outages between deployments.
- Keep scheduled monitors small and fast — a handful of critical journeys, not the full suite.
- Send failures to a Slack/Teams channel or email so someone is alerted rather than discovering it in the next build.
- Postman Monitors are a hosted alternative if you do not want to own the scheduling in CI.

Common Mistakes

- Committing real tokens or passwords inside the exported environment file instead of injecting them as secrets.
- Masking the exit code so the pipeline stays green even when tests fail.
- Letting the collection and environment JSON in the repo drift out of sync with what is in the Postman app.
- Producing no artifact, so a failed CI run gives you no way to see what actually happened.
- Running the entire suite as a synchronous deploy gate when only a fast smoke set is needed.

INSIDE STLC PRO TIP

Export your collection and environment into the repo and run them with Newman in CI from day one. The moment your API checks live in the pipeline instead of one person's Postman app, they become a real quality gate the whole team can trust.